

GSoC 2026 Proposal

Project Title

Mobile-First Feature Parity for Open Food Facts Explorer Through Shared Data Layer and Reusable OFF Components

Candidate Profile and Contribution Evidence

I am an active Open Food Facts Explorer contributor with recent contributions in accessibility, API behavior correctness, and UX stability. I have already worked through real review cycles and maintainer feedback.

Contribution evidence:

- ? <https://github.com/openfoodfacts/openfoodfacts-explorer/pull/1230>
- ? <https://github.com/openfoodfacts/openfoodfacts-explorer/pull/1109>
- ? <https://github.com/openfoodfacts/openfoodfacts-explorer/pull/1100>
- ? <https://github.com/openfoodfacts/openfoodfacts-explorer/pull/1083>
- ? <https://github.com/openfoodfacts/openfoodfacts-explorer/pull/1058>
- ? <https://github.com/openfoodfacts/openfoodfacts-explorer/issues/1157>
- ? <https://github.com/openfoodfacts/openfoodfacts-explorer/issues/1106>
- ? <https://github.com/openfoodfacts/openfoodfacts-explorer/issues/1098>

Problem Statement

Open Food Facts Explorer still has parity gaps in critical mobile user journeys and repeated API integration logic across views. This causes user friction and slows contributor velocity due to duplicated request/response handling and inconsistent state behavior.

This project addresses both product quality and maintainability by improving three committed high-impact flows and introducing a shared typed data-access layer.

Committed Scope (Exactly 3 Flows)

1. Search -> Product -> Back to Search state continuity.
2. Product quick-understanding section for mobile-first readability.
3. Basic edit flow with stable folksonomy autosuggest behavior.

Implementation surfaces:

- ? src/routes/search
- ? src/routes/products/[barcode]
- ? src/lib/ui/edit-product-steps

Out of scope:

- ? Full parity for all legacy pages
- ? Full visual redesign
- ? Global migration of all endpoints

Why These Flows

- ? Mobile is the dominant usage context; these are high-frequency mobile journeys.

- ? They align with maintainer priorities: parity, reliability, and maintainability.
- ? Flow 3 is included because on-the-go edits are common on phones; stale suggestions directly reduce edit trust and completion.

Technical Plan

- ? Introduce shared typed data-access helpers for the three committed flows.
- ? Use openfoodfacts-nodejs SDK where endpoint coverage exists.
- ? Use thin custom adapters only for endpoint gaps or route-specific data mapping.
- ? Standardize loading, error, and empty states across targeted flows.
- ? Integrate at least two reusable OFF webcomponents where reuse reduces complexity.

Deliverables and Acceptance Criteria

Deliverable A: Shared typed data layer

- ? Adopted in all three committed flows.
- ? Error and loading states standardized in modified surfaces.
- ? Duplicated API integration logic reduced by at least 30 percent in targeted modules.
- ? SDK-first implementation documented with adapter boundaries.

Deliverable B: 3 improved high-impact flows

- ? No functional regressions on mobile and desktop for the three flows.
- ? Accessibility checks pass for newly introduced interactions.
- ? Each flow includes at least one quantified before/after improvement metric.

Flow-specific acceptance:

- ? Flow 1: query, filter, sort, pagination, and scroll context restored on return to search.
- ? Flow 2: key product facts become visible earlier on mobile without extra navigation.
- ? Flow 3: no stale autosuggest overwrite during rapid input changes.

Deliverable C: Reusable OFF webcomponents

- ? At least two OFF webcomponents integrated in production-facing paths.
- ? Integration and customization notes documented for contributors.

Deliverable D: Maintainer handoff package

- ? Architecture note for shared data layer.
- ? Migration checklist for extending the pattern.
- ? Worked migration example.
- ? Final KPI report with before/after evidence.

Baseline and Target Metrics

Flow 1: Search -> Product -> Back

- ? Baseline method: 20 manual mobile runs with state-restore matrix.
- ? Target: at least 95 percent successful state restoration.

Flow 2: Product quick understanding

- ? Baseline method: timed task on fixed product set in mobile viewport.
- ? Target: at least 30 percent reduction in median time to identify key facts.

Flow 3: Basic edit with autosuggest

- ? Baseline method: scripted rapid-input interaction tests and manual verification.
- ? Target: zero stale autosuggest updates in validation runs.

Timeline (175 Hours)

Community Bonding

- ? Confirm scoped surfaces and acceptance criteria with mentor.
- ? Capture baseline metrics and lock reporting template.
- ? Finalize review cadence.

Weeks 1 to 3

- ? Implement shared data-layer slice.
- ? Migrate Flow 1.
- ? Open small reviewable PRs.

Weeks 4 to 6

- ? Migrate Flows 2 and 3.
- ? Integrate OFF webcomponents.
- ? Complete accessibility and responsive validations.

Weeks 7 to 9

- ? Reliability and performance refinements.
- ? Documentation and handoff package.
- ? Final KPI report.

Risk Management

- ? Scope creep: strict cap at three committed flows.
- ? API inconsistency: isolate transformations in adapters.
- ? External API instability: robust loading and error states with deterministic fallback behavior.

Communication Plan

- ? Weekly progress update in #off-explorer.
- ? Incremental PR strategy for early maintainer feedback.
- ? Scope changes only with impact note and fallback plan.

Definition of Done

- ? Three committed flows merged and verified on mobile and desktop.
- ? Shared typed data layer used in all targeted surfaces.
- ? KPI before/after evidence included in final report.
- ? Architecture note, migration checklist, and worked example delivered.
- ? At least one full maintainer review iteration addressed.

Project Information

- ? Repository: <https://github.com/openfoodfacts/openfoodfacts-explorer>
- ? Related repositories: <https://github.com/openfoodfacts/openfoodfacts-nodejs>,
<https://github.com/openfoodfacts/openfoodfacts-webcomponents>
- ? Slack: #off-explorer

? Potential mentor: VaiTon

? Duration: approximately 175 hours

? Skills: TypeScript, Svelte/SvelteKit, HTTP APIs

? Difficulty: Medium